worst-case output $\mathbf{y}^\star = \min \mathcal{R}^*(\mathbf{y}|\tilde{\mathbf{x}}_{1:n})$. Thus, we have the following set of equations to describe the generation process:

$$\mathbf{y}^\star = \min \mathcal{R}^*(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) \tag{2}$$

$$\mathcal{L}^{adv}(\tilde{\mathbf{x}}_{1:n}) = -\log p(y^\star|\tilde{\mathbf{x}}_{1:n}). \tag{3}$$

$$\tilde{\mathbf{x}}_{1:n} = \underset{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})}{\operatorname{argmin}} \mathcal{L}^{adv}(\tilde{\mathbf{x}}_{1:n}), \tag{4}$$

where $\mathcal{A}(\hat{\mathbf{x}}_{1:n})$ is the distribution or set of possible jailbroken instructions. Note that this encompasses *all* possible adversarial prompt modifications within the maximum prompt length. All attacks under our threat model eventually come down to ways to minimize Eq. 3.

3.2 Defense Objective

While prevailing methods to improve LLM alignment involve fine-tuning, the objective of matching human preferences does not generally account for adversaries and jailbreaking. In addition, the high cost of generating attack prompts makes standard adversarial training on these samples difficult [Jain et al., 2023]. We center our optimization approach on the *prompt* level to address this. We formalize this as the negative log probability of a target token output $\mathbf{y}'$ that refuses $\tilde{\mathbf{x}}_{1:n}$. This can be represented as the *normal output* of an LLM trained to maximize $\mathcal{R}'$ or $\mathbf{y}' = \max \mathcal{R}^*(\mathbf{y}|\tilde{\mathbf{x}}_{1:n})$. Thus, we have the following safe loss and defense objective

$$\mathbf{y}' = \max \mathcal{R}^*(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) \tag{5}$$

$$\mathcal{L}^{safe}(\tilde{\mathbf{x}}_{1:n}) = -\log p(\mathbf{y}'|\tilde{\mathbf{x}}_{1:n}) \tag{6}$$

$$\text{minimize}\, \mathcal{L}^{safe}(\tilde{\mathbf{x}}_{1:n}). \tag{7}$$

The goal of the defense objective is to ensure robustness even under worst-case scenarios, such as when a jailbreak alters the harmful prompt. Since $\tilde{\mathbf{x}}_{1:n}$ is generated through Eq. 4, this can be formalized by incorporating the adversary into Eq. 7, which yields the following objective,

$$\text{minimize}\, \mathcal{L}^{safe}(\underset{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})}{\operatorname{argmin}} \mathcal{L}^{adv}(\tilde{\mathbf{x}}_{1:n})) \tag{8}$$

Eq. 8 directly incorporates Eq. 4 and like adversarial training, this formulation can be viewed as the composition of two problems, an *inner minimization* problem, and *outer minimization* problem. Jailbreaking can be interpreted as optimizing the inner minimization problem by creating a prompt to minimize the adversarial loss while existing defenses implicitly optimize the outer minimization problem. In contrast, we propose the first method to optimize the overall objective directly.

3.3 Robust Prompt Optimization

Without direct gradient updates to the LLM, we focus on input optimization, which is challenging for LLMs due to the discreteness of text. We use gradient-based token optimization, which can directly optimize for Eq. 8. Gradient-based optimization is especially useful in our setting, as harmless behavior has well-defined output targets described in Eq. 6. In general, solving this objective means creating a mapping between *any* worst-case modification of the input or jailbreak and the distribution of aligned output responses under the original prompt. This can be achieved by optimizing a suffix or set of trigger tokens that is always followed by a harmless response. To do so, we propose our main algorithm, *Robust Prompt Optimization (RPO)*, which optimizes for a set of tokens to enforce this mapping. As a whole, RPO consists of two successive steps based on the two components of the overall objective: (1) a jailbreak generation and selection step that applies a worst-case modification to the prompt and (2) a discrete optimization step that modifies the suffix to maintain refusal.

We simulate the adaptive threat model for the first step by adding the current defensive suffix to the original prompt and applying or generating a jailbreak prompt afterward. This is a straightforward modification to the prompt for simple, manual jailbreaks such as in-context examples [Wei et al., 2024]. For automatic jailbreaks such as GCG [Zou et al., 2023], we apply several iterations of the jailbreak until either the RPO suffix is broken or until a fixed compute budget is exhausted. This allows RPO to support a variety of attacks during the optimization process. Our main technical contribution for this component is the selection step, where after its respective generation, we apply

Algorithm 1 Robust Prompt Optimization

Require: Prompts $x^{(1)}_{1:n_1} \dots x^{(m)}_{1:n_m}$, set of jailbreaks $\mathcal{A}$, initial defensive suffix $p_{1:l}$, losses $\mathcal{L}^{\text{safe}}_1 \dots, \mathcal{L}^{\text{safe}}_m$, iterations T, k, batch size B, selection interval R

for $s = 1, \dots, S$ **do**
  loop T times
    for all prompts $x^{(1)}_{1:n_1} \dots x^{(m)}_{1:n_m}, j = 1 \dots m$ **do**
      Append defensive suffix $p_{1:l}$ to $x^{(j)}_{1:n_i}$
      if $t \mod R == 0$ **then** ▷ *Apply selection every R steps*
        $A^* := \text{argmin}_{\mathcal{A}} \mathcal{L}^{\text{adv}}_j \sum_{1 \le o \le m} (A_o(x^{(j)}))$ ▷ *Jailbreak that minimizes adv. loss*
        $x^{(j)} := A^*(x^{(j)})$ ▷ *Apply best jailbreak from set to prompt*
    for $i \in [0 \dots l]$ **do**
      $\mathcal{X}_i := \text{Top-}k(-\sum_{1 \le j \le m} \nabla_{e_{p_i}} \mathcal{L}^{\text{safe}}_j(x^{(j)}_{1:n+l} \| p_{1:l}))$ ▷ *Compute top-k candidates*
    for $b = 1, \dots, B$ **do**
      $\tilde{p}^{(b)}_{1:l} := \text{Uniform}(\mathcal{X}_i)$ ▷ *Sample replacements*
    $p_{1:l} := \tilde{p}^{(b^\star)}_{1:l}$, where $b^\star = \text{argmin}_b \sum_{1 \le j \le m} \mathcal{L}^{\text{safe}}_j(x^{(j)}_{1:n+l} \| \tilde{p}^{(b)}_{1:l})$ ▷ *Best replacement*
return Optimized defensive suffix p

the jailbreak prompt that minimizes the adversarial loss for that instruction, according to Eq. 4. As the adversarial loss is calculated with the addition of the current RPO suffix, this ensures the optimization is performed under worst-case conditions and reduces the chance for the suffix to overfit on a particular jailbreak. In practice, due to the cost of generating new attack prompts, we only perform this operation again after a certain number of iterations R.

After a jailbreak is applied, the second step optimizes the suffix to minimize the safe loss Eq. 7. We adopt a method similar to AutoPrompt [Shin et al., 2020] and GCG, using a greedy coordinate descent approach to assess how replacing the *i*-th token affects the safe loss. This involves calculating the first-order approximation and selecting the top-*k* tokens with the largest negative gradient. We then randomly select $B \le k|\mathcal{I}|$ tokens from this set of candidates, obtain the exact loss on this subset, and replace the current token with the token with the smallest loss. Both steps are applied in succession for a number of iterations T. The full algorithm is described in Alg. 1.

3.4 Theoretical Analysis of RPO

In this section, we provide a theoretical analysis to characterize the effectiveness and robustness properties of RPO under various settings. We study how the performance of RPO is affected when applied to different instruction datasets and against unknown adversaries.

Setup. We first introduce the notations and setup used in the analysis. Let $\mathbf{X}$ denote a benchmark dataset and $\mathbf{P}_{\mathcal{X}}$ be the underlying data distribution. We simplify Obj. 8 based on reward model $\mathcal{R}$:

$$\max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

where $\mathcal{A}(\hat{\mathbf{x}}_{1:n})$ represents the set of possible adversarial transformations for prompt $\hat{\mathbf{x}}_{1:n}$. The attack success rate (ASR) of an adversary τ on dataset $\mathbf{X}$ is defined as:

$$\text{ASR}(\mathbf{X})_\tau = 1 - \sum_{\hat{\mathbf{x}}_{1:n} \in \mathbf{X}} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

We denote RPO optimized against adversary τ as $\gamma(\tau)$. The ASR of $\gamma(\tau)$ on dataset $\mathbf{X}$ is defined as:

$$\text{ASR}(\mathbf{X})_{\gamma(\tau)} = 1 - \sum_{\hat{\mathbf{x}}_{1:n} \in \mathbf{X}} \max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

To measure the effectiveness of RPO, we define $\text{Diff}(\mathbf{X}, \gamma(\tau), \tau)$ as the difference in ASR between the original model and the model defended by RPO:

$$\text{Diff}(\mathbf{X}, \gamma(\tau), \tau) = \sum_{\hat{\mathbf{x}}_{1:n} \in \mathbf{X}} \max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) - \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

Performance on the Same Dataset, Known Adversary. We first consider the case where RPO is applied to the same dataset and adversary it was optimized on.

Proposition 3.1.

$$\text{Diff}(\mathbf{X}, \gamma(\tau), \tau) \geq 0$$

This proposition states that when RPO is applied to the same dataset it was optimized on and the adversary is known, it is guaranteed to reduce ASR.

Generalization to Different Datasets, Known Adversary Next, we study the generalization performance of RPO when applied to datasets sampled from the underlying data distribution $\mathbf{P}_{\mathcal{X}}$. We extend the notation of Diff to the distribution setting:

$$\text{Diff}(\mathbf{P}_{\mathcal{X}}, \gamma(\tau), \tau) = \mathbb{E}_{\hat{\mathbf{x}}_{1:n} \sim \mathbf{P}_{\mathcal{X}}} \max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) - \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

Lemma 3.2. *Let n denote the number of samples in $\mathbf{X}$. The expected effectiveness of RPO on samples from $\mathbf{P}_{\mathcal{X}}$ is bounded as follows:*

$$\mathbb{P}\left(\text{Diff}(\mathbf{X}, \gamma(\tau), \tau) - \text{Diff}(\mathbf{P}_{\mathcal{X}}, \gamma(\tau), \tau) \geq \epsilon\right) \leq \exp(-2n\epsilon^2).$$

This lemma bounds the generalization error of RPO when applied to samples from the underlying data distribution. It shows that the effectiveness of RPO on the training dataset $\mathbf{X}$ is close to its expected effectiveness on the entire data distribution $\mathbf{P}_{\mathcal{X}}$, with high probability.

Performance on the Same Dataset, Unknown Adversary In practice, the adversary and attacks encountered during testing may differ from the ones used during optimization. We denote the training time adversary as τ and the unknown test time adversary as ζ. We use $\mathcal{A}_\tau$ and $\mathcal{A}_\zeta$ to represent the adversarial perturbations generated by τ and ζ, respectively.

We study $\text{Diff}(\mathbf{X}, \gamma(\tau), \zeta)$, defined as:

$$\text{Diff}(\mathbf{X}, \gamma(\tau), \zeta) = \sum_{\hat{\mathbf{x}}_{1:n} \in \mathbf{X}} \max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\tau(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) - \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\zeta(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

Proposition 3.3. *With n denoting the number of samples in $\mathbf{X}$, we have*

$$\text{Diff}(\mathbf{X}, \gamma(\tau), \zeta) \geq \text{Diff}(\mathbf{X}, \gamma(\tau), \tau) + \frac{1}{n} \sum_{\hat{\mathbf{x}}_{1:n} \in \mathbf{X}} \mathbb{I}\left[\min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\zeta(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) < \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\tau(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n})\right].$$

This proposition compares the empirical strength of the two adversaries τ and ζ. If τ is empirically stronger than ζ on dataset $\mathbf{X}$, then $\text{Diff}(\mathbf{X}, \gamma(\tau), \zeta) \geq \text{Diff}(\mathbf{X}, \gamma(\tau), \tau)$. This means that RPO optimized against a stronger adversary τ will still be effective against a weaker test time adversary ζ. However, if ζ is stronger than τ, the effectiveness of RPO may degrade, and the degradation depends on the empirical difference in strength between the two adversaries.

Generalization to Different Datasets, Unknown Adversary Finally, we study the generalization performance of RPO when applied to datasets from $\mathbf{P}_{\mathcal{X}}$ and against an unknown adversary ζ. We define $\text{Diff}(\mathbf{P}_{\mathcal{X}}, \gamma(\tau), \zeta)$ as:

$$\text{Diff}(\mathbf{P}_{\mathcal{X}}, \gamma(\tau), \zeta) = \mathbb{E}_{\hat{\mathbf{x}}_{1:n} \sim \mathbf{P}_{\mathcal{X}}} \max_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}(\hat{\mathbf{x}}_{1:n})} \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\tau(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) - \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\zeta(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

Theorem 3.4. *Let n denote the number of samples in $\mathbf{X}$, and $p_{\zeta,\tau}$ be the probability that adversary ζ is stronger than τ on samples from $\mathbf{P}_{\mathcal{X}}$, i.e.,*

$$\min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\zeta(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}) < \min_{\tilde{\mathbf{x}}_{1:n} \in \mathcal{A}_\tau(\hat{\mathbf{x}}_{1:n})} \mathcal{R}(\mathbf{y}|\tilde{\mathbf{x}}_{1:n}).$$

Then, with probability at least $1 - \delta$, we have

$$\text{Diff}(\mathbf{P}_{\mathcal{X}}, \gamma(\tau), \zeta) \geq \text{Diff}(\mathbf{X}, \gamma(\tau), \tau) - \sqrt{\frac{1}{2n} \log\left(\frac{1}{\delta}\right)} + np_{\zeta,\tau}.$$